

INPUT	OUTPUT
<pre> #include <stdio.h> #include <stdlib.h> #include <ctype.h> #include <string.h> // Node structure for Expression Tree typedef struct TreeNode { char data; struct TreeNode *left; struct TreeNode *right; } TreeNode; // Node structure for Stack of Tree Nodes typedef struct StackNode { TreeNode* treeNode; struct StackNode* next; } StackNode; // Node creation helper TreeNode* createTreeNode(char data) { TreeNode* node = (TreeNode*)malloc(sizeof(TreeNode)); node->data = data; node->left = NULL; node->right = NULL; return node; } // Stack Operations void push(StackNode** top, TreeNode* treeNode) { StackNode* newNode = (StackNode*)malloc(sizeof(StackNode)); newNode->treeNode = treeNode; newNode->next = *top; *top = newNode; } TreeNode* pop(StackNode** top) { if (*top == NULL) return NULL; StackNode* temp = *top; TreeNode* poppedNode = temp- >treeNode; *top = (*top)->next; free(temp); </pre>	<pre> --- Direct Stack Postfix Evaluation --- Result: 11 --- Expression Tree Construction & Traversals --- In-order Traversal: ((8+(3*2))-(6/2)) Pre-order Traversal: - + 8 * 3 2 / 6 2 Post-order Traversal: 8 3 2 * + 6 2 / - --- Expression Tree Evaluation --- Result: 11 </pre>

```

    return poppedNode;
}

// Stack-based Evaluation Array Stack
Implementation
int evaluatePostfixStack(const char* exp) {
    int stack[100];
    int top = -1;

    for (int i = 0; exp[i] != '\0'; i++) {
        if (exp[i] == ' ') continue;

        if (isdigit(exp[i])) {
            stack[++top] = exp[i] - '0';
        } else {
            int val2 = stack[top--];
            int val1 = stack[top--];
            switch (exp[i]) {
                case '+': stack[++top] = val1 + val2;
                break;
                case '-': stack[++top] = val1 - val2;
                break;
                case '*': stack[++top] = val1 * val2;
                break;
                case '/': stack[++top] = val1 / val2;
                break;
            }
        }
    }
    return stack[top];
}

// Build Expression Tree
TreeNode* buildExpressionTree(const char*
postfix) {
    StackNode* stack = NULL;

    for (int i = 0; postfix[i] != '\0'; i++) {
        if (postfix[i] == ' ') continue;

        TreeNode* node =
createTreeNode(postfix[i]);
        if (isdigit(postfix[i])) {
            push(&stack, node);
        } else {
            node->right = pop(&stack);
            node->left = pop(&stack);

```

```

        push(&stack, node);
    }
}
return pop(&stack);
}

// Evaluate Expression Tree
int evaluateExpressionTree(TreeNode* root)
{
    if (!root) return 0;

    if (!root->left && !root->right) {
        return root->data - '0';
    }

    int leftVal = evaluateExpressionTree(root->left);
    int rightVal =
evaluateExpressionTree(root->right);

    switch (root->data) {
        case '+': return leftVal + rightVal;
        case '-': return leftVal - rightVal;
        case '*': return leftVal * rightVal;
        case '/': return leftVal / rightVal;
    }
    return 0;
}

// Traversals
void inorder(TreeNode* root) {
    if (root) {
        if (root->left || root->right) printf("(");
        inorder(root->left);
        printf("%c", root->data);
        inorder(root->right);
        if (root->left || root->right) printf(")");
    }
}

void preorder(TreeNode* root) {
    if (root) {
        printf("%c ", root->data);
        preorder(root->left);
        preorder(root->right);
    }
}

```

```

void postorder(TreeNode* root) {
    if (root) {
        postorder(root->left);
        postorder(root->right);
        printf("%c ", root->data);
    }
}

int main() {
    char postfix[] = "832*+62/-";

    printf("--- Direct Stack Postfix Evaluation -\n");
    int resultStack =
evaluatePostfixStack(postfix);
    printf("Result: %d\n\n", resultStack);

    printf("--- Expression Tree Construction & Traversals ---\n");
    TreeNode* root =
buildExpressionTree(postfix);

    printf("In-order Traversal: ");
    inorder(root);
    printf("\n");

    printf("Pre-order Traversal: ");
    preorder(root);
    printf("\n");

    printf("Post-order Traversal: ");
    postorder(root);
    printf("\n\n");

    printf("--- Expression Tree Evaluation ---\n");
    int resultTree =
evaluateExpressionTree(root);
    printf("Result: %d\n", resultTree);

    return 0;
}

```